CS1231S Midterms Cheatsheet
by @jmsandiegoo

1. Proofs

Number System Sets

$\mathbb{N}$: Natural numbers $\{0, 1, 2, ...\}$ ($\mathbb{Z}_{\geq 0}$)
$\mathbb{Z}$: Integers $\{3, -100, 3^5, ...\}$
$\mathbb{Q}$: Rational Numbers $\{\frac{1}{2}, -23, 8.6, \frac{-37}{5}, ...\}$
$\mathbb{R}$: Real Numbers $\{-1, \pi, \sqrt{2}, 4.5, ...\}$
** 0 is neither negative or positive*
** $\mathbb{Z}^{+/-}$ set of all positive or negative integers*

Terminologies

Axiom/Postulate: A statement assumed to be true w/o proof. Basic building blocks of theorems
Theorem: A mathematical statement proved using rigorous mathematical reasoning. Major / important result.
Lemma: A small theorem minor result to help prove a theorem.
Corollary: Simple deduction from Theorem.
Conjecture: A statement believed to be true, but no proof yet.

Basic Properties of Integers

Closure: Closed under $+/\times/-$
$x + y \in \mathbb{Z}$ and $xy \in \mathbb{Z}$
Commutativity: Commutative under $+/\times$
$x + y = y + x$ and $xy = yx$
Associativity: Associative under $+/\times$
$x + y + z = (x + y) + z = x + (y + z)$ and $xyz = (xy)z = x(yz)$
Distributivity:
$x(y + z) = xy + xz$
Trichotomy: Exactly one of the following is true
$x = y$ or $x < y$ or $x > y$

Definitions of Number

Even and Odd Integers

Let $n \in \mathbb{Z}$
n is even $\leftrightarrow \exists k \in \mathbb{Z}\ n = 2k$
n is odd $\leftrightarrow \exists k \in \mathbb{Z}\ n = 2k + 1$

Divisibility

Let $d, n \in \mathbb{Z} \wedge d \neq 0$
$d|n \leftrightarrow \exists k \in \mathbb{Z}\ n = dk$
** d divides n*

Prime / Composite

Rationality / Lowest Term Fraction

r is rational $\leftrightarrow \exists a, b \in \mathbb{Z}\ r = \frac{a}{b} \wedge b \neq 0$
** it is lowest term fraction if largest integer that divides $a \wedge b$ in $\frac{a}{b}$ is 1 (prime numbers)*

Methods of Proof

1. **Proof by Construction**: usually proves an existential statement by explicitly finding / constructing a value that satisfies the properties stated.
2. **Proof by Counter Example**: Finding a specific example that shows statement is not always true.
3. **Proof by Exhaustion**: Prove by showing each cases is true. Suitable for finite cases.
4. **Proof by Deduction**: Form of direct proof usually makes use of definitions. Good for universal statements and infinite cases.
5. **Proof by Contradiction**: Negating the original statement and trying to arrive at a contradiction making the negation to be false.

2. Compound Statements

Definition 2.1.1 (Statement)
A statement / proposition is a sentence that is true or false, but not both
Definition 2.1.2 (Negation)
If p is a statement variable, negation is "not p ($\sim p$)"
Definition 2.1.3 (Conjunction)
"p and q ($p \wedge q$)"
Definition 2.1.4 (Disjunction)
"p or q ($p \vee q$)"
** $\sim$ performed first, then $\wedge$, $\vee$ coequal in order of operation*
Definition 2.1.5 (Statement / Propositional Form) A statement / propositional form is an expression of statement variables and logical connectives that becomes a statement when actual statements are substituted
Definition 2.1.6 (Logical Equivalence)
$P \equiv Q \leftrightarrow$ both have identical truth values
** Use truth table / counter example to prove non-equivalence*
Definition 2.1.7 (Tautology)
Statement that is always true (tautological statement)
Definition 2.1.8 (Contradiction)
Statement that is always false (contradictory statement)

Conditional Statements

Definition 2.2.1 (Conditional)
"if p then q" or "p implies q"
$p\ (hypo/antedecent) \rightarrow q\ (conclusion/consequent)$
** only false when p is true and q is false*
** performed last with $\sim, \wedge, \vee$*
Vacuously true: hypothesis is false
Implication: $\sim p \vee q$
Negation: $p \wedge \sim q$
Definition 2.2.2 (Contrapositive)
"$p \rightarrow q \equiv \sim q \rightarrow \sim p$"
Definition 2.2.3 (Converse)
"$q \rightarrow p$"
Definition 2.2.4 (Inverse)
"$\sim p \rightarrow \sim q$"
** Inverse $\equiv$ Converse*
Definition 2.2.5 (Only If)
p can only take place only if q takes place. "$\sim q \rightarrow \sim p$"
Definition 2.2.6 (Biconditional $\leftrightarrow$)
"$p \leftrightarrow q$" implication but both ways. True when p and q has same value false otherwise.
** Proving it needs to show both direction (Proof by cases $\rightarrow$ and $\leftarrow$)*
** Coequal with Conditional operator*
Definition 2.2.7 (Necessary / Sufficient)
"$p \rightarrow q$" means p is sufficient to guarantee occurrence of q.
"$\sim p \rightarrow \sim q$" means if p does not occur, q does not either. Necessary for p to occur when q occurs

Arguments

Definition 2.3.1 (Argument)

$p \rightarrow q$	(premise)
p	(premise)
$\bullet\ q$	(conclusion)

Valid: All premises (critical rows) and conclusion are true.
Syllogism: Argument form consisting of two premises and a conclusion.

Rules of inference

Modus Ponens

$p \rightarrow q$	(premise)
p	(premise)
$\bullet\ q$	(conclusion)

Modus Tollens

$p \rightarrow q$	(premise)
$\sim q$	(premise)
$\bullet\ \sim p$	(conclusion)

Generalization

p	(premise)
$\bullet\ p \vee q$	(conclusion)

Specialization

$$p \wedge q \quad \text{(premise)}$$
$$\bullet \quad p \quad \text{(conclusion)}$$

Elimination

$$p \vee q \quad \text{(premise)}$$
$$\sim q \quad \text{(premise)}$$
$$\bullet \quad p \quad \text{(conclusion)}$$

Transitivity

$$p \to q \quad \text{(premise)}$$
$$q \to r \quad \text{(premise)}$$
$$\bullet \quad p \to r \quad \text{(conclusion)}$$

Div into Cases

$$p \vee q \quad \text{(premise)}$$
$$p \to r \quad \text{(premise)}$$
$$q \to r \quad \text{(premise)}$$
$$\bullet \quad r \quad \text{(conclusion)}$$

Contradiction

$$\sim p \to false \quad \text{(premise)}$$
$$\bullet \quad p \quad \text{(conclusion)}$$

Fallacies

1. Converse
2. Inverse
3. False Premise

Definition 2.3.2 (Sound / Unsound Arguments)
Sound $\leftrightarrow$ if it is valid and premises are true
Unsound $\leftrightarrow$ $\sim$ Sound

3. Quantified Statements

Definition 3.1.1 (Predicate): is a sentence that contains a finite number of variables that becomes a statement when values are substituted
Definition 3.1.2 (Truth Set): set of all elements in Domain of P(x) such that P(x) is true
Definition 3.1.3 (Universal Statement $\forall$):
"$\forall x \in D, Q(x)$"
** true $\leftrightarrow$ $Q(x) \equiv$ True for every x in D*
** false $\leftrightarrow$ $Q(x) \equiv$ False for at least one x in D (counter-example)*
Definition 3.1.4 (Existential Statement $\exists$):
"$\exists x \in D, Q(x)$"
** true $\leftrightarrow$ $Q(x) \equiv$ True for at least one x in D*
** false $\leftrightarrow$ $Q(x) \equiv$ False for all x in D (counter-example)*
** $\exists!$: there exists a unique / one and only one*
Theorem 3.2.1 Negation of a Universal Statement:
"$\sim(\forall x \in D, P(x)) \equiv \exists x \in D, \sim P(x)$"
Theorem 3.2.2 Negation of a Existential Statement:
"$\sim(\exists x \in D, P(x)) \equiv \forall x \in D, \sim P(x)$"
Definition 3.2.1 (Contrapositive, converse, inverse):
Given "$\forall x \in D(P(x) \to Q(x))$"

1. Contrapositive: "$\forall x \in D(\sim Q(x) \to \sim P(x))$"
2. Converse: "$\forall x \in D(Q(x) \to P(x))$"
3. Invsrse: "$\forall x \in D(\sim P(x) \to \sim Q(x))$"

Proven, Theorems etc.

Numbers

Product of two consecutive odd numbers is always odd
The product of two irrational numbers is not always irrational
Difference of two consecutive sqaures is always odd
Theorem 4.7.1: $\sqrt{2}$ is irrational
Proposition 4.6.4: For all integers n, if n^2 is even then n is even

Quantifiers

Theorem 3.2.1 Negation of Universal Statement

- **Byte:** 8-bits
- **Nibble:** 4-bits
- **Word:** 1 / 2 / 4 bits etc. based on architecture

$Nbits = 2^N values$
$Mvalues = log_2 M$ *take ceiling value*

Decimal (base 10)

$Symbols = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9$

Binary (base 2)

$Symbols = 0, 1$
** represented by prefix 0b*

Octal (base 8)

$Symbols = 0, 1, 2, 3, 4, 5, 6, 7$
** represented by prefix 0*

Hexadecimal (base 16)

$Symbols = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F$
** represented by prefix 0x*

Conversions

- Base-R to Decimal
  - multiply each digit with respective positional weight. $(digit \times R^{position}) + ...$
- Decimal to Base-R
  - **whole number:** Repeated division by R
    ** to find remainder multiply fractional value with the divisor / R*
  - **fractions:** Repeated multiplication by R Method.
    ** Until fractional product is 0 or desired number of decimal decimal places*
- Between Bases
  - partition in groups of $currR^N = convertR$

ASCII

American Standar Code for Information Interchange
7 bits plus 1 bit for odd / even parity (for error checking)
Integers (0 - 127) are interchangeable with Characters

Negative / Signed Numbers

- **Sign-and-Magnitude**
  - Sign is represented by the MSB '0' is positive
  - Range for 8-bit:
    $-2^{n-1} - 1 = -127_{10}$ *to* $2^{n-1} - 1 = +127_{10}$
  - Two zeroes: $00000000 = +0_{10}$ & $10000000 = -0_{10}$
- **1s Complement**
  - Negated value of x: $-x = 2^n - x - 1$ / Flip the bits
  - Range for 8-bit:
    $-2^{n-1} - 1 = -127_{10}$ *to* $2^{n-1} - 1 = +127_{10}$
  - Two zeroes: $00000000 = +0_{10}$ & $11111111 = -0_{10}$
- **2s Complement**
  - Negated value of x: $-x = 2^n - x$ / $1s\ complement + 1$ / Copy from LSB until first '1' then invert rest till MSB
  - Range for 8-bit:
    $-2^{n-1} = -128_{10}$ *to* $2^{n-1} - 1 = +127_{10}$
  - One zero: $00000000 = +0_{10}$
- **Excess Representation**
  - Range of values distributed evenly between positive and negative values
  - Excess-M starts from 00..00 $= -M$ onwards
  - To evenly distribute, use the -2^{n-1} range as M value

Addition / Subtraction 1s & 2s Complement

- **1s Complement**
  1. Perform binary Addition
  2. Carry out add 1 to result
  3. Check overflow if MSB result is opposite sign of A & B
- **2s Complement**
  1. Perform binary Addition
  2. Ignore carry out of MSB
  3. Check overflow 'carry in/out' of MSB different / like in 1s complement

Fixed-Point Representation

Number of bits allocated for whole and fractional part are fixed For example: If 2s complement is used:
$011010.11_{2s} = 26.75_{10}$
$111110.11_{2s} = -000001.01_2 = -1.25_{10}$
Limitations: limited range and precision and immutable once implemented

Floating-Point Representation (IEE 754)

Allows to represent very large or small numbers copared to fixed point such as:
0.23×10^{23} (very large positive number)
0.5×10^{-37} (very small positive number)
-0.2397×10^{-18} (very small negative number)

IEE 754 Single precision (32-bits)

Consists of 3 main components:

- Sign (1-bit)
  - 0/1: Positive / Negative
- Exponent (8-bit w/ excess-127)
  - $exponent + 127 = 129 = 10000001_2$
- Mantissa (fraction) (23-bit)
  - nomralised with an implicit leading bit 1
  - $110.1_2 \rightarrow 1.101_2 \times 2^2$ (101 is stored in mantissa)

11. Processor Datapath

MIPS Instruction Execution

Fetch Stage

1. Fetch the instruction from memory with **Program Counter (PC)**
   ** PC is a special register*
2. Increment PC by 4 to get next address
3. Output is provided to Decode / Operand Fetch Stage using the Instruction Memory. ** address → binary instruction*

PC is read during first half of clock period and updated at next rising clock edge

Decode / Operand Fetch Stage

Gather data from instruction field:

1. Input is the binary instruction from Fetch Stage
2. Read opcode to determine instruction format and field lengths
3. Fetch data from necessary registers into the Write Register according to RegDst Signal (Rt / Rd)
4. Output is the necessary operands to be conducted in ALU Stage

Multiplexer

Selects a specific input from multiple input lines. Consists of:

- Inputs: n lines of same width (32-bits)
- Control: m bits where $n = 2^m$. Specifies which input to choose
- Output: The selected input based on i^{th} input is $control = i$

ALU Stage / Execution Stage

Performs the following operations:

- Arithmetic, Shifting & Logical
- Memory operation - address calculation
- Branch operaton - register comparison & target address calculation

1. Input is the two 32-bits numbers from Decode Stage 2nd Operand is chosed according to ALUSrc signal
2. With the help of 4-bit ALUControl signal, the respective operation is conducted
3. Outputs ALUresult & a 1-bit signal isZero

Branching in ALU

1. Identify the branch outcome through isZero to handle equal / not equal
2. Calculate Branch address with PC and Offset (Immediate value)
3. Output will be used by PCSrc signal

Memory Stage

Mainly for memory read and write instructions **lw / sw**. Other instructions remains idle in this stage and instead ALU result passed into Register Write Stage

1. Input computation result to be used as Memory Address. Also Write Data for sw (Register Data 2)
2. Read or write data on the memory address depending on MemRead/Write Signal
3. Output memory address content to Register Write stage (lw)

Register Write Stage

Routes the correct result into the Write Data in register file. Instructions like stores, branches and jumps remains idle in this stage.

1. Input computation result from Memory or ALU, which is specified by MemToReg signal
2. Outputs the selected result into the Write Register in the register file

12. Processor Control

Control Signals

- **RegDst** - (@Decode/OperandFetch) Selects the destination of WriteRegister in register file
  - 0/1: WriteRegister = Ins[20:16] (rt) / Inst[15:11] (rd / immediate)
- **RegWrite** - (@Decode/OperandFetch) Indicates if writing on WriteRegister is enabled
  - 0/1: No register write / WD write to WR
- **ALUSrc** - (@ALU) Determines the 2nd ALU input
  - 0/1: Read Data 2 / SignExtend(Inst15:0) (Sign extended Immediate)
- **ALUcontrol** - (@ALU) Determines the type of operation to execute
  - 0000: AND
  - 0001: OR
  - 0010: add
  - 0110: subtract
  - 0111: slt
  - 1100: NOR
- **MemRead** - (@Memory) Indicates if read from memory using Address returned by ALU
  - 0/1: No read / Reads memory with Address returned by ALU
- **MemWrite** - (@Memory) Indicates if write to memory using RD2 of Register File
  - 0/1: No memory write / Writes RD2 into the memory in Address returned by ALU
- **MemToReg** - (@RegWrite) Indicates which one to write back into WriteData in register file
  - 0/1: WriteData = ALU result / Memory read data
- **PCSrc** - (@ALU) Indicates whether to take branch address or not
  - 0/1: $PC = PC + 4$ / $PC = SignExt(Inst[15:0]) << 2 + (PC + 4)$
  - PCSrc = isBranch AND isZero

ALU

** Note: operation is determined by the last 2-bits **
Subtraction has Cin = 1 for bit-0

Multilevel Decoding

Used to generate the ALUcontrol signal. *(Simpler and more efficient design than Brute Force approach)*

1. Generate a 2-bit ALUop signal from opcode
2. Use ALUop and Function code (for R-type) to generate 4-bit ALUcontrol signal

Control Combinational Circuit

Instruction Execution

How the implementation of fetch, decode, memory, write, etc. are actually done
Mainly three things:

1. Read contents from register / memory
2. Perform some form of computation
3. Write results to register / memory

Single Cycle Execution

All performed within a clock period.
disadvantage: all instructions will take the same time as the slowest one.

Multi Cycle Execution

Break up instructions into execution steps (Fetch, Decode, ALU Operation, Mem read/write, Register Write). Each execution is one clock cycle. **disadvantage:** time taken depends on instructions and may not necessarily be faster.

Datapath & Control Diagrams